Strategic Architecture and Execution Plan for Sui DeFi Protocol Engineering

Interrogation of the Target Engineering Profile

An exhaustive analysis of Emmanouil "Manos" Liolios's professional footprint reveals a highly sophisticated engineering profile defined by a commitment to full-stack decentralized architecture, rigorous capability-based security patterns, and collaborative development frameworks. His active contributions within the Mysten Labs organization, coupled with his public advocacy for specific design paradigms, provide a clear blueprint for the technical competencies he values in a senior protocol engineer.[1] Approaching an interview with him requires an understanding of not just the Move programming language, but the holistic lifecycle of deploying, testing, and securing applications on the Sui network.

The GitHub metrics associated with the target interviewer present a developer who prioritizes rapid, iterative, and highly collaborative workflows. Earning the "Pair Extraordinaire" achievement multiple times indicates a strong preference for pair programming and cross-functional code reviews.[1] Furthermore, his status as a "Pull Shark" and "Quickdraw" suggests that he operates efficiently within continuous integration and continuous deployment (CI/CD) pipelines, favoring modular, easily reviewable pull requests over monolithic code drops.[1] Demonstrating an understanding of modern version control, modular package management, and continuous testing will align perfectly with his expectations for a Staff-Level Engineer.

A critical aspect of his engineering focus is his full-stack orientation. Unlike protocol engineers who focus exclusively on smart contract logic, his repositories demonstrate a unified approach connecting modern front-ends directly to Sui smart contracts. For instance, his suizino project leverages a Vue3 front-end to interface with on-chain logic.[1] Additionally, his instrumental role in developing the Move Registry (MystenLabs/mvr) utilizing TypeScript, Rust, and Move underscores a deep appreciation for developer tooling and ecosystem infrastructure.[1] The Move Registry project was designed to replace hardcoded, 64-character hexadecimal object addresses with human-readable, verifiable names to improve dependency management within Programmable Transaction Blocks (PTBs).[3] The MVR system ensures that developers can call the latest version of a smart contract without manually updating package addresses across their front-end codebase.[2] Consequently, utilizing the MVR TypeScript SDK plugin to resolve dynamic package addresses during client-side PTB construction will serve as an immediate, high-impact demonstration of ecosystem alignment during a technical evaluation.[3]

The target's engineering philosophy is heavily indexed on advanced access control mechanisms, specifically the native Sui Move "Capability" pattern. During his presentation at Sui Basecamp 2024, he articulated a definitive framework for structuring on-chain authorization.[6]

He emphasized that access control should never rely on hardcoded administrative addresses—a common anti-pattern in Solidity and Solana development—but rather on explicitly issued Capability objects.[6] He advocates for deploying a universally accessible Shared Object for public interactions while restricting sensitive state mutations through functions that require a specific Capability object passed as a reference.[6] Moving beyond simple binary access keys, he recommends embedding verifiable logic directly into the Capability struct. He specifically advocates for quota-based capabilities that restrict usage amounts and time-based capabilities that enforce strict expiration epochs to secure backend game servers and automated agents.[6] Furthermore, he highlighted the use of native multi-signature setups and on-chain quorums, utilizing Sui's object-wrapping properties to create parent "wrapper" objects that require community votes before a high-stakes Capability can be unwrapped and utilized.[6]

Demonstrating mastery of these specific patterns—particularly combining them with the fast-path execution of Owned Objects versus the Mysticeti consensus requirements of Shared Objects—will establish profound technical credibility. To earn immediate respect during a code walkthrough, a candidate must shift the conversation away from Solana's Program Derived Addresses (PDAs) and Cross-Program Invocation (CPI) callbacks, proving that the candidate has fully internalized the security benefits of the Move Bytecode Verifier and the Object-Centric Data Model.

Advanced Project Architectures

The following three project architectures are engineered to be executed within a 48-hour development window while strictly demonstrating the core paradigms of the Sui execution model: Object-Centric Data design, Programmable Transaction Blocks, advanced Move security patterns, and DeepBook V3 integration. These projects are designed to act as strategic talking points, showcasing an evolution from Solana's account-based constraints to Sui's dynamic object orchestration.

Project Architecture 1: Atomic Flash-Arbitrage Engine with Hot Potato Sequencing

The first project concept is a highly optimized, intent-based arbitrage executor that leverages DeepBook V3 flash loans and a secondary decentralized exchange, mathematically secured against reentrancy via Move's Hot Potato pattern orchestrated entirely within a single PTB. This architecture directly challenges the vulnerabilities inherent in Ethereum and Solana flash loan designs, presenting a superior paradigm for decentralized finance.

The system relies on a hybrid data model to maximize throughput and minimize latency. The primary object is the ArbitrageurCap, an Owned Object assigned directly to the bot operator. Because it is owned by a single address, transactions invoking this capability bypass the Mysticeti DAG consensus protocol entirely. Instead, they utilize Byzantine Consistent Broadcast

to achieve execution speeds via the fast path, avoiding the latency associated with global state coordination.[9] Conversely, the DeepBook V3 Pool and the BalanceManager are strictly Shared Objects. The DeepBook Pool comprises three internal components: the Book, the State, and the Vault.[11] The Book component manages the core order book logic and matching engine; the State component maintains individual user accounting data, historical volumes, and governance parameters; and the Vault serves as the settlement layer, finalizing the movement of base and quote tokens.[11] Because multiple market participants interact with the Pool simultaneously, the transaction must be sequenced through the Mysticeti consensus layer to prevent state conflicts and ensure strict causal ordering.[13]

The Programmable Transaction Block acts as the unified orchestration layer, executing a complex sequence atomically without the need for an intermediary smart contract to hold the funds mid-flight.[15] The PTB begins by calling borrow_flashloan_base on the DeepBook V3 Pool. This function returns two distinct items: a Coin<BaseAsset> representing the borrowed liquidity, and a FlashLoan receipt object.[17] The PTB then routes the borrowed Coin into a custom swap function on a secondary automated market maker, yielding a Coin<QuoteAsset>. This newly acquired asset is immediately routed back into DeepBook V3 via the swapExactQuoteForBase function, aiming to yield a larger quantity of the original base asset.[18] Finally, the PTB invokes return_flashloan_base, passing the required repayment amount alongside the FlashLoan receipt. The DeepBook contract consumes the receipt, satisfying the compiler, and the PTB splits the remaining profit, transferring it directly to the operator's address.[17]

This architecture inherently relies on the Hot Potato pattern. The FlashLoan receipt is defined as a struct completely devoid of the key, store, drop, or copy abilities.[17] Because it lacks the drop ability, it cannot be discarded or ignored by the execution context; because it lacks store or key, it cannot be placed into global persistent storage.[17] The Move Bytecode Verifier mathematically guarantees that the transaction will fail at compile-time if the receipt is not explicitly unpacked and destroyed by the originating lending protocol's repayment function.[17]

During the code walkthrough, the primary talking point will focus on the architectural superiority of the Hot Potato pattern over Solana's callback-based execution. On Solana, an arbitrageur must rely on CPIs and instruction introspection to verify flash loan repayment. This forces the lending program to yield the execution context to the borrower's program, and upon return, the lending program must verify account state deltas to ensure the funds were returned.[21] This reliance on pre- and post-execution account state checks introduces severe reentrancy vulnerabilities and requires complex validation logic.[23] On Sui, the asset is passed dynamically as a tangible object, and the Hot Potato mathematically guarantees repayment at the language layer, rendering runtime balance checks and reentrancy guards entirely obsolete.[17]

Architectural Vector	Solana Account-Based Flash Loan	Sui Move Object-Centric Flash Loan
Execution Model	Instruction introspection and CPI callbacks.	Object passing and sequential PTB execution.
Security Enforcement	Runtime validation of account balance deltas.	Compile-time guarantee via structural abilities.
Reentrancy Risk	High; requires explicit reentrancy guard state locks.	Non-existent; Hot Potato must be consumed linearly.
State Complexity	Requires locking all read/write accounts upfront.	Dynamic object passing handles state autonomously.
Development Overhead	Demands extensive defensive programming.	Secure by default through type system constraints.

Project Architecture 2: Time-Locked Vault with Delegated AI Trading Capabilities

The second project concept is a non-custodial asset management vault that mints time-bound, quota-limited capabilities, allowing an automated AI agent or backend service to execute trades on DeepBook V3 without possessing custody of the underlying assets. This directly implements the security paradigms favored by the target interviewer for backend infrastructure integration.

The core of this architecture is the Vault, instantiated as a Shared Object to hold the user's capital in the form of a Balance<T> rather than a Coin<T>, preventing accidental transfers.[25] The access control is fragmented into two distinct Owned Objects: the OwnerCap and the DelegatedTradingCap. The user retains the OwnerCap, granting absolute authority to withdraw funds, upgrade the contract, or revoke delegation entirely. The automated trading agent holds the DelegatedTradingCap.[25] By isolating the authorization logic into Owned Objects, the

system minimizes state contention. When the agent initiates a trade, the validators quickly verify the ownership of the DelegatedTradingCap via the fast-path before queuing the actual asset swap through the Mysticeti consensus required by the DeepBook V3 Shared Object.[9]

The PTB flow for this architecture acts as a seamless pipeline. The transaction begins by passing the DelegatedTradingCap and a mutable reference to the Vault into a custom authenticate_trade function. The Move logic asserts that the current epoch is less than the capability's expiration epoch and that the requested trade volume does not exceed the internal quota embedded in the capability struct.[6] Upon successful assertion, the function extracts a specific Coin object from the Vault's internal Balance.[25] The extracted Coin is piped directly into DeepBook V3's swapExactBaseForQuote function within the same PTB.[25] The resulting output coins—the requested base asset, any leftover quote asset, and excess DEEP tokens used for fees—are routed back into the Vault's internal balance through a deposit function.[25]

This architecture heavily utilizes the Capability pattern with embedded internal state, directly mirroring the target interviewer's specific recommendations for securing backend systems where high-speed transaction signing is required.[6] By placing a u64 representing the expiration epoch and a u64 representing the remaining trade volume directly inside the DelegatedTradingCap struct, the protocol enforces strict execution boundaries without requiring additional storage lookups.[6] Furthermore, the architecture relies on strict resource typing; the capability is granted the key and store abilities to allow it to be held by the agent's address and transferred if necessary, but it explicitly lacks the copy ability, ensuring the trading rights cannot be maliciously duplicated by a compromised backend.[20]

To impress the interviewer, the dialogue should highlight the profound contrast between Sui's Object Capability model and Solana's Program Derived Addresses (PDAs) for access control. On Solana, building a delegated trading system requires deriving a PDA to act as a cryptographic signing authority and storing the delegation rules within a discrete data account linked to that PDA via seeds.[27] This account-centric model tightly couples access logic to specific program addresses, making it exceedingly difficult to transfer trading rights or revoke them without executing complex account data mutations and managing rent exemptions.[27] On Sui, a Capability is a tangible, distinct object.[29] Delegation is achieved simply by minting the object and transferring it to the agent. Revocation can be executed universally by utilizing a shared version-control object to invalidate the specific capability instance, or by simply having the user burn the parent capability.[6]

Authorization Metric	**Solana PDA & Data Account Model**	**Sui Move Capability Object Model**

Identity Verification	Based on cryptographic seed derivation and signing.	Based on definitive ownership of a distinct struct.
Rights Transferability	Complex; requires rewriting account data and authorities.	Trivial; managed via native transfer::public_transfer.
State Embedded Logic	Requires separate state accounts linked to the PDA.	State variables live directly inside the Capability struct.
Revocation Mechanism	Closing the data account and recovering rent.	Updating a version registry or destroying the object.

Project Architecture 3: Quorum-Gated DeepBook V3 Liquidity Provisioning

The third project concept is a multi-signature, institutional-grade market-making system utilizing object-wrapping to ensure that DeepBook V3 limit orders and DEEP token staking require threshold consensus from multiple authorized actors before execution.

This project utilizes a LiquidityVault configured as a Shared Object. Access to manage the vault's deep liquidity is gated by a QuorumCap. However, the QuorumCap is not an Owned Object assigned to a single user; it is a Wrapped Object permanently hidden inside a VotingBooth Shared Object.[6] Individual institutional participants hold a MemberCap, which is an Owned Object. The Mysticeti consensus mechanism guarantees the precise ordering of votes applied to the VotingBooth by ensuring causal-ordering of the shared state.[9] Once a mathematical threshold of votes is reached within a specific epoch, the QuorumCap is unwrapped and temporarily utilized within the execution context.

The PTB flow demonstrates the raw composability of the Sui network. Multiple signers generate cryptographic signatures off-chain via a custom interface. A single relayer constructs a massive PTB, importing the various MemberCap objects as inputs alongside the signatures. The PTB executes a tally_votes function on the VotingBooth. If the required threshold is met, the function extracts and returns the QuorumCap as a temporary variable.[6] The PTB then utilizes the QuorumCap to authorize a call to DeepBook V3. The transaction can submit a new limit order to the Book component, or call stake on the State component to lock DEEP tokens.[12] Staking DEEP tokens in a DeepBook V3 pool provides the institution with governance rights,

allowing them to propose changes to taker fees, maker fees, and stake requirements, while also making them eligible for maker rebates.[11] Finally, the PTB ensures the QuorumCap is immediately returned to the VotingBooth to be wrapped and secured for the next epoch of voting.[6]

The core patterns utilized here are Object Wrapping and Dynamic Field management. Because Sui objects can natively own other objects, complex hierarchical data structures can be created securely.[33] The QuorumCap is isolated and protected by the type system until the wrapping struct explicitly permits its extraction, preventing rogue actors from accessing the liquidity pool. Dynamic Fields can be attached to the VotingBooth to track individual proposal states without bloating the core struct size.[35]

When comparing this architecture to Solana, the discussion must center on transaction composability and the orchestration of multiple signers. On Solana, a multi-signature transaction requires a heavily serialized execution flow where participants must sign discrete transactions that interact with a central multi-sig program, which then executes a CPI to the target protocol.[33] Solana's rigid Compute Unit constraints often fragment complex, multi-step DAO executions, forcing developers to build intermediate state accounts to bridge transactions.[36] Sui's Programmable Transaction Blocks natively allow up to 1,024 sequential operations within a single execution block.[16] A PTB can dynamically load multiple MemberCap objects, execute the quorum logic, unwrap the capability, interact with the DeepBook V3 matching engine, manage the DEEP token staking, and finalize the transaction in one atomic, seamless action, backed by a unified gas payment model.[15]

Execution Metric	**Solana Multi-Sig LP Architecture**	**Sui Move Quorum PTB Architecture**
Transaction Structure	Fragmented across multiple serialized transactions.	Atomic execution within a single 1,024-command PTB.
Composability	Reliant on deeply nested Cross-Program Invocations.	Sequential, declarative command chaining on the client side.
Compute Constraints	Strict Compute Unit limits per transaction block.	Bound by a massive, unified gas budget per PTB.

Data Hierarchy	Flat account space relying on relational mapping.	True object wrapping and nested hierarchical ownership.

The Build Execution Plan: 48-Hour Sprint

Successfully deploying a high-fidelity proof-of-concept within a single weekend requires absolute discipline and strict adherence to a structured execution pipeline. The following framework is designed to take the project from basic scaffolding to deterministic simulation testing and final deployment, mirroring the exact workflows utilized by elite protocol engineers at Mysten Labs.

Phase 1: Environment Architecture and Scaffolding

The foundation of the project must mirror production-grade Mysten Labs environments, specifically incorporating tools favored by the target interviewer. This phase ensures the local development environment is perfectly calibrated for Sui Move development.

The process begins with rigorous toolchain validation. The developer must ensure the sui CLI is installed and configured to the correct network environment, specifically targeting the Sui Testnet where the DeepBook V3 liquidity pools are actively deployed and seeded. The Rust toolchain must be verified, and the IDE should be instantiated with the move-analyzer extension to provide strict, real-time type checking and borrow-checker validations. Package initialization is executed via the command sui move new flash_arbitrage, which generates the standard Move directory structure encompassing the Move.toml manifest, the sources/ directory for core logic, and the tests/ directory for validation.[38]

Dependency injection requires modifying the Move.toml to include the Sui standard library and the official DeepBook V3 package repository. To directly appeal to the target interviewer's specific contributions to the ecosystem, the architecture must integrate the Move Registry (MVR). Instead of tracking raw hex addresses for PTB construction, the developer must install the @mysten/mvr-static plugin for the TypeScript SDK.[2] This preparation allows client-side calls to route through human-readable MVR names (e.g., @mysten/deepbook-v3), demonstrating an advanced understanding of Sui package management and an appreciation for the infrastructure tools the interviewer personally built.[3]

Phase 2: Smart Contract Synthesis and Deterministic Testing

The primary focus of this phase is translating the architectural design into Move bytecode, relying heavily on the Sui Framework's rigorous testing paradigms. This phase moves beyond simple syntax and focuses on enforcing invariants at the compiler level.

Struct and Capability definitions form the bedrock of the logic. The core objects must be defined with absolute precision regarding their abilities. The ArbitrageurCap is established with key and store abilities, allowing it to exist in global storage and be transferred between addresses. The internal Vault is defined as a shared object utilizing Balance<T> rather than Coin<T> for secure internal accounting, preventing the exposure of transferable token objects within the shared state.[25] DeepBook interface mapping requires implementing the interaction logic with the DeepBook V3 Pool. The developer writes wrapper functions that invoke borrow_flashloan_base and return_flashloan_base.[17] The function signatures must be meticulously designed to properly ingest and output the FlashLoan Hot Potato receipt without accidentally consuming it incorrectly.

Testing on Sui requires a paradigm shift from traditional local validators. Mysten Labs heavily prioritizes deterministic simulation testing methodologies, often referred to as simtest.[39] Originating from architecture like FoundationDB, deterministic simulation testing runs the software in a fully controlled environment where all sources of non-determinism—such as network calls and system time—are mocked.[41] In the Move environment, this is achieved by initializing a test module using #[test_only] and utilizing sui::test_scenario.[42] This framework mocks the multi-transaction flow across different epoch states and user addresses, allowing the developer to simulate advanced consensus conditions locally.

Crucially, the testing phase must include Negative Testing for the Hot Potato pattern. The developer must write a specific test case that deliberately attempts to ignore the FlashLoan receipt or attempts to grant it the drop ability.[17] Executing this test will cause the Move compiler to explicitly catch the missing consumption parameter, failing the build. This failure is the ultimate success metric; it proves to the interviewer the absolute, mathematical absence of reentrancy vectors, showcasing that security is enforced at the language level rather than relying on the developer to remember to add runtime checks.[17]

Phase 3: Client-Side PTB Orchestration

The architectural elegance of Sui lies in the fact that complex orchestration happens on the client side, mitigating the need for bloated, gas-heavy smart contracts. This phase involves building the JavaScript/TypeScript infrastructure to construct the transaction block.

SDK initialization requires setting up a Node.js environment importing @mysten/sui/transactions and @mysten/deepbook-v3.[3] MVR global registration is the next critical step. By registering the MVR plugin globally within the application context, the transaction block dynamically resolves the latest package upgrades from the registry without manual code refactoring.[3]

Transaction block construction begins by instantiating const tx = new Transaction();.[3] The developer chains a series of commands to create the atomic flow. First, a moveCall command is appended to borrow the base asset, destructuring the outputs to capture both the coin and the flash_loan_receipt.[18] Next, an intermediate swap command is appended, utilizing the output

coin from the previous step as the input parameter.[15] Finally, the terminating moveCall is appended to return the flash loan, explicitly passing the flash_loan_receipt back into the DeepBook V3 protocol.[18] To demonstrate elite optimization skills, the developer ensures the PTB leverages native gas smashing to automatically consolidate dust coins, optimizing the overall gas expenditure of the 1,024-command capacity.[44]

Phase 4: Final Deployment and Presentation Formatting

The final phase ensures the project is robust, deployed, and formatted into a compelling narrative for the technical interview.

Before committing state changes, the developer performs dry run executions. Utilizing the sui client ptb --dry-run command or the SDK's native dry-run simulation, the developer estimates gas limits and verifies the atomic execution paths.[45] This ensures that the complex arbitrage logic will not fail due to unexpected dynamic field resolutions or insufficient gas budgets. Once validated, network publication is executed. The package is published to the Sui Testnet using sui client publish --gas-budget <AMOUNT>. The developer captures the resulting Package ID and Upgrade Capability objects, locking them into the MVR registry to provide a clean, human-readable endpoint for the interview.[46]

Walkthrough preparation is the final, decisive step. The developer structures the code presentation to immediately highlight the absence of traditional reentrancy guards, shifting the focus to the Hot Potato struct definition. The narrative must trace the lifecycle of the capabilities as they move through the fast-path for sub-second validation, and articulate exactly when the execution hands over to the Mysticeti DAG for the shared state modifications within the DeepBook V3 Vault.[9] By presenting a perfectly executed PTB that leverages human-readable MVR routing, Hot Potato security guarantees, and deterministic testing proofs, the candidate will unequivocally demonstrate the architectural mastery required for a Staff-Level engineering position at Mysten Labs.

Works cited

1. Manolis Liolios manolisliolios - GitHub, accessed March 5, 2026, https://github.com/manolisliolios
2. MystenLabs/mvr: The repository which holds the mvr project. - GitHub, accessed March 5, 2026, https://github.com/MystenLabs/mvr
3. Transaction Plugin for Sui TypeScript SDK | SuiNS Documentation, accessed March 5, 2026, https://docs.suins.io/move-registry/tooling/typescript-sdk
4. mvr/README.md at main · MystenLabs/mvr - GitHub, accessed March 5, 2026, https://github.com/MystenLabs/mvr/blob/main/README.md
5. The Move Registry (MVR) | The Sui Stack - Medium, accessed March 5, 2026, https://medium.com/the-sui-stack/the-move-registry-mvr-195b0c55a780
6. Building access control on Sui using "capabilities" | Manos Liolios at Sui Basecamp 2024, accessed March 5, 2026, https://www.youtube.com/watch?v=nIAUkllpMwI

7. Move Smart Contract Audit Checklist - Hacken.io, accessed March 5, 2026, https://hacken.io/discover/move-smart-contract-audit-checklist/
8. Move for Solidity Developers I: Storage and Access Control - CertiK, accessed March 5, 2026, https://www.certik.com/resources/blog/move-for-solidity-developers-part-1-storage-and-access-control
9. Comparison - Sui Documentation, accessed March 5, 2026, https://docs.sui.io/sui-compared
10. Consensus - Sui Documentation, accessed March 5, 2026, https://docs.sui.io/concepts/sui-architecture/consensus
11. MystenLabs/deepbookv3: Deepbook V3 - GitHub, accessed March 5, 2026, https://github.com/MystenLabs/deepbookv3
12. Design | Sui Documentation, accessed March 5, 2026, https://docs.sui.io/standards/deepbookv3/design
13. Mysticeti Set to Supercharge Consensus on Sui, accessed March 5, 2026, https://blog.sui.io/mysticeti-consensus-reduce-latency/
14. SUI vs Ethereum vs Solana: A Next-Gen Blockchain Comparison - XBTFX, accessed March 5, 2026, https://xbtfx.com/article/sui-vs-ethereum-vs-solana
15. sui-move-intro-course/unit-five/lessons/1_programmable_transaction_block.md at main, accessed March 5, 2026, https://github.com/sui-foundation/sui-move-intro-course/blob/main/unit-five/lessons/1_programmable_transaction_block.md
16. Composability Through Different Lenses: PTBs and EIP-7702 - The Sui Blog, accessed March 5, 2026, https://blog.sui.io/composability-ptb-eip7702/
17. How Sui Move rethinks flash loan security - The Trail of Bits Blog, accessed March 5, 2026, https://blog.trailofbits.com/2025/09/10/how-sui-move-rethinks-flash-loan-security/
18. Flash Loans SDK - Sui Documentation, accessed March 5, 2026, https://docs.sui.io/standards/deepbookv3-sdk/flash-loans
19. Pattern: Hot Potato | The Move Book, accessed March 5, 2026, https://move-book.com/programmability/hot-potato-pattern/
20. The Ability Mistakes That Will Drain Your Sui Move Protocol | by Pushkar Mishra - Medium, accessed March 5, 2026, https://medium.com/@pushkarm029/the-ability-mistakes-that-will-drain-your-sui-move-protocol-1a2c317e373f
21. Flash Loan Basics (Balancer) | Flashbots Docs, accessed March 5, 2026, https://docs.flashbots.net/flashbots-mev-share/searchers/tutorials/flash-loan-arbitrage/flash-loan-basics
22. Strengthening DeFi Security: A Static Analysis Approach to Flash Loan Vulnerabilities, accessed March 5, 2026, https://arxiv.org/html/2411.01230v2
23. Understanding Flash Loan Attacks Through a Practical Example | by Seyyed Ali Ayati, accessed March 5, 2026, https://medium.com/@seyyedaliayati/understanding-flash-loan-attacks-through-a-practical-example-b2f9644e6012

24. Web3 underlying language (2): How does Move avoid flash loan reentrancy attacks? | 链捕手ChainCatcher on Binance Square, accessed March 5, 2026, https://www.binance.com/en/square/post/141511
25. Suistody | ETHGlobal, accessed March 5, 2026, https://ethglobal.com/showcase/suistody-rx5a8
26. Swaps | Sui Documentation, accessed March 5, 2026, https://docs.sui.io/standards/deepbookv3-sdk/swaps
27. Solana - Sui Documentation, accessed March 5, 2026, https://docs.sui.io/concepts/sui-for-solana
28. Sui vs Solana: Comprehensive Blockchain Comparison & Analysis - Ledger, accessed March 5, 2026, https://www.ledger.com/academy/topics/blockchain/sui-vs-solana
29. Move for Solidity Developers: Token Standard II — Advanced Fungible Token Extensions, accessed March 5, 2026, https://www.certik.com/resources/blog/move-for-solidity-developers-token-standard-ii-advanced-fungible-token
30. Breaking the Mold: Sui, a Revolutionary L1 | by Jennifer Barnett | Oregon Blockchain Group, accessed March 5, 2026, https://medium.com/oregon-blockchain-group/breaking-the-mold-sui-a-revolutionary-l1-5f72e625661a
31. Staking and Governance SDK - Sui Documentation, accessed March 5, 2026, https://docs.sui.io/standards/deepbookv3-sdk/staking-governance
32. Deepbook (DEEP) White paper In accordance with Title II of Regulation (EU) 2023/1114 (MiCA) - Kraken, accessed March 5, 2026, https://assets-cms.kraken.com/files/51n36hrp/facade/5e2072afc89b939d8c6fc140ea7f396911e9bc6c.pdf
33. Compare Solana's Transaction Lifecycle & Sui's Object Runtime - Helius, accessed March 5, 2026, https://www.helius.dev/blog/solana-vs-sui-transaction-lifecycle
34. Sui Network: Demystifying the Monolithic Contender - Delphi Digital, accessed March 5, 2026, https://members.delphidigital.io/reports/sui-network-demystifying-the-monolithic-contender
35. Solana vs Sui: Pros and Cons - Backpack Learn, accessed March 5, 2026, https://learn.backpack.exchange/articles/solana-vs-sui-pros-and-cons
36. What if the #1 lending protocol on Sui launched a DEX? | Four Pillars, accessed March 5, 2026, https://4pillars.io/en/articles/what-if-the-top-lending-protocol-on-sui-launched-a-dex
37. Programmable Transaction Blocks (PTBs) - Sui Documentation, accessed March 5, 2026, https://docs.sui.io/guides/developer/transactions/prog-txn-blocks
38. How to write tests for Move on Sui smart contracts | by Jude Miracle | Medium, accessed March 5, 2026, https://medium.com/@judemiracle/how-to-write-tests-for-move-on-sui-smart-contracts-8e70bd00e0f4
39. walrus/CLAUDE.md at main - GitHub, accessed March 5, 2026,

https://github.com/MystenLabs/walrus/blob/main/CLAUDE.md
40. Deterministic simulation testing | Antithesis, accessed March 5, 2026, https://antithesis.com/resources/deterministic_simulation_testing/
41. SE Radio 685: Will Wilson on Deterministic Simulation Testing, accessed March 5, 2026, https://se-radio.net/2025/09/se-radio-685-will-wilson-on-deterministic-simulation-testing/
42. Sui Move Scenario Testing, accessed March 5, 2026, https://forums.sui.io/t/sui-move-scenario-testing/27870
43. deepbook-trading | Skills Marketplace - LobeHub, accessed March 5, 2026, https://lobehub.com/pt-BR/skills/randypen-sui-eco-skills-deepbook-trading
44. Sui Programmable Transaction Block (PTBs) | Learning Move 0x03 - NonceGeekDAO, accessed March 5, 2026, https://noncegeek.medium.com/sui-programmable-transaction-block-ptbs-learning-move-0x03-6b1abef44dea
45. Deepsentinel | ETHGlobal, accessed March 5, 2026, https://ethglobal.com/showcase/deepsentinel-i6koq
46. Sui Smart Contracts with Move: A Brutally Honest Introduction to Blockchain's Most Misunderstood Language - Paul Serban, accessed March 5, 2026, https://www.paulserban.eu/blog/post/sui-smart-contracts-with-move-a-brutally-honest-introduction-to-blockchains-most-misunderstood-language/
47. manolisliolios/suizino: A small casino-like running on Sui Blockchain. Front-end made with Vue3. - GitHub, accessed March 5, 2026, https://github.com/manolisliolios/suizino